

程式碼研究室 - 使用 Firestore、Vector Search、Langchain 和 Gemini 建構情境式瑜珈姿勢推薦應用程式 (Node.js 版本)

來源：<https://codelabs.developers.google.com/yoga-pose-firestore-vectorsearch-nodejs?hl=zh-tw>

步驟數：10

本程式碼研究室將引導您建立知識驅動的瑜珈姿勢推薦應用程式，該應用程式會提供相符的瑜珈姿勢，回答使用者的問題。您將瞭解如何從 Hugging Face 資料集建立瑜珈姿勢的 Firestore 集合、設定 Firestore 向量搜尋，並將所有內容整合至 Node.js 應用程式。

目錄

1. [簡介](#)
2. [事前準備](#)
3. [設定 Firestore](#)
4. [準備 Yoga 姿勢資料集](#)
5. [將資料匯入 Firestore 並生成向量嵌入](#)
6. [將完整瑜珈姿勢匯入 Firestore 資料庫集合](#)
7. [在 Firestore 中執行向量相似度搜尋](#)
8. [網頁應用程式](#)
9. [\(選用\) 部署至 Google Cloud Run](#)
10. [恭喜](#)

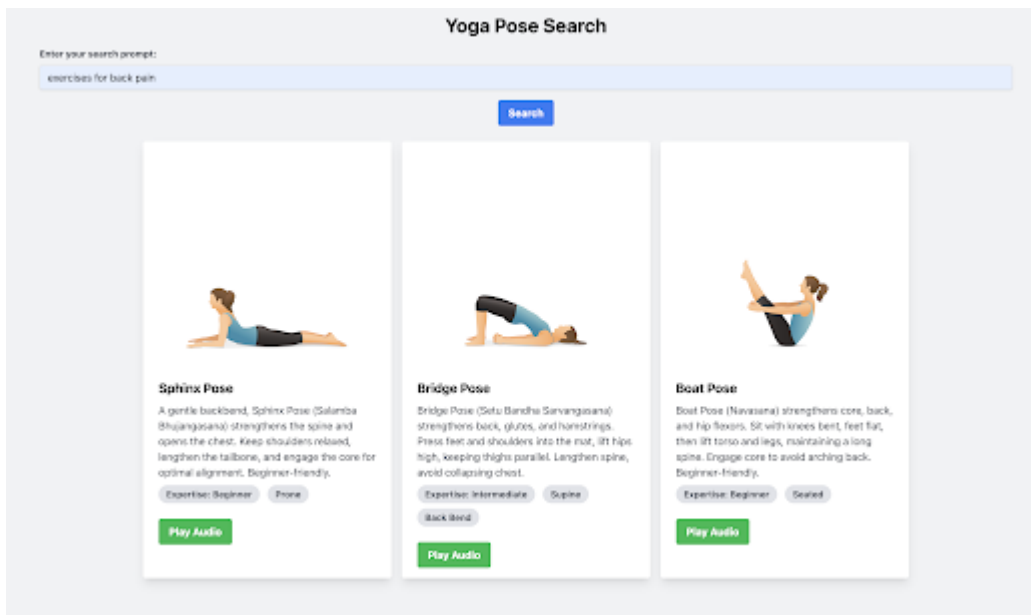
步驟 1 · 約 5 分鐘

1. 簡介

在本程式碼研究室中，您將建構應用程式，使用向量搜尋功能推薦瑜珈姿勢。

在本程式碼研究室中，您將逐步完成下列步驟：

1. 使用現有的 Hugging Face 瑜珈姿勢資料集 (JSON 格式)。
2. 使用 Gemini 為每個姿勢生成說明，並新增欄位說明來強化資料集。
3. 將瑜珈姿勢資料載入為 Firestore 集合中的文件集合，並產生嵌入內容。
4. 在 Firestore 中建立複合式索引，以啟用向量搜尋功能。
5. 在 Node.js 應用程式中使用向量搜尋，將所有內容整合在一起，如下所示：



執行步驟

- 設計、建構及部署網頁應用程式，運用 Vector Search 推薦瑜珈體式。

課程內容

- 如何使用 Gemini 生成文字內容，以及在本程式碼研究室的脈絡中，生成瑜珈姿勢的說明
- 如何從 Hugging Face 的強化資料集載入記錄，並連同向量嵌入一起匯入 Firestore
- 如何使用 Firestore 向量搜尋功能，根據自然語言查詢搜尋資料
- 如何使用 Google Cloud Text-to-Speech API 產生音訊內容

軟硬體需求

- Chrome 網路瀏覽器
- Gmail 帳戶
- 已啟用計費功能的 Cloud 專案

本程式碼研究室適合各種程度的開發人員 (包括初學者)，並在範例應用程式中使用 JavaScript 和 Node.js。不過，您不需要具備 JavaScript 和 Node.js 知識，也能瞭解本文介紹的概念。

2. 事前準備

建立專案

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境，預先載入了 bq。點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。



4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

〇〇〇〇

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

〇〇〇〇

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 透過下列指令啟用必要的 API。這可能需要幾分鐘的時間，請耐心等待。

```
gcloud services enable firestore.googleapis.com \
    compute.googleapis.com \
    cloudresourcemanager.googleapis.com \
    servicenetworking.googleapis.com \
    run.googleapis.com \
    cloudbuild.googleapis.com \
    cloudfunctions.googleapis.com \
    aiplatform.googleapis.com \
    texttospeech.googleapis.com
```

成功執行指令後，您應該會看到類似下方的訊息：

```
Operation "operations/..." finished successfully.
```

除了使用 gcloud 指令，您也可以透過控制台搜尋各項產品，或使用這個[連結](#)。

如果遺漏任何 API，您隨時可以在導入過程中啟用。

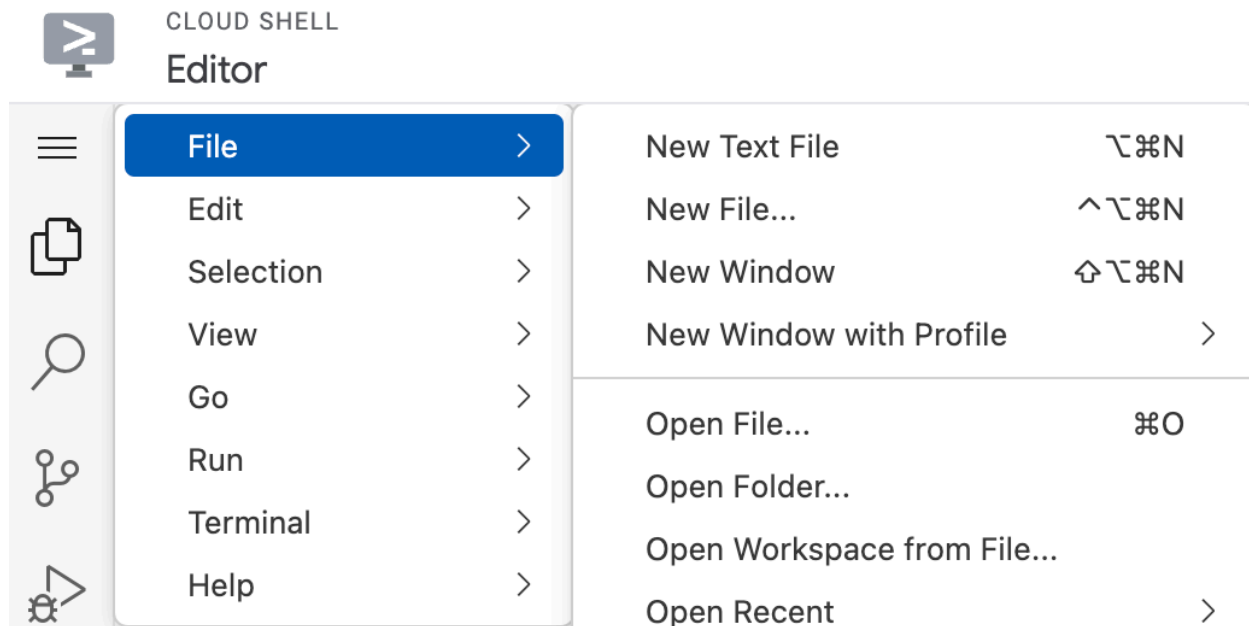
如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

複製存放區並設定環境設定

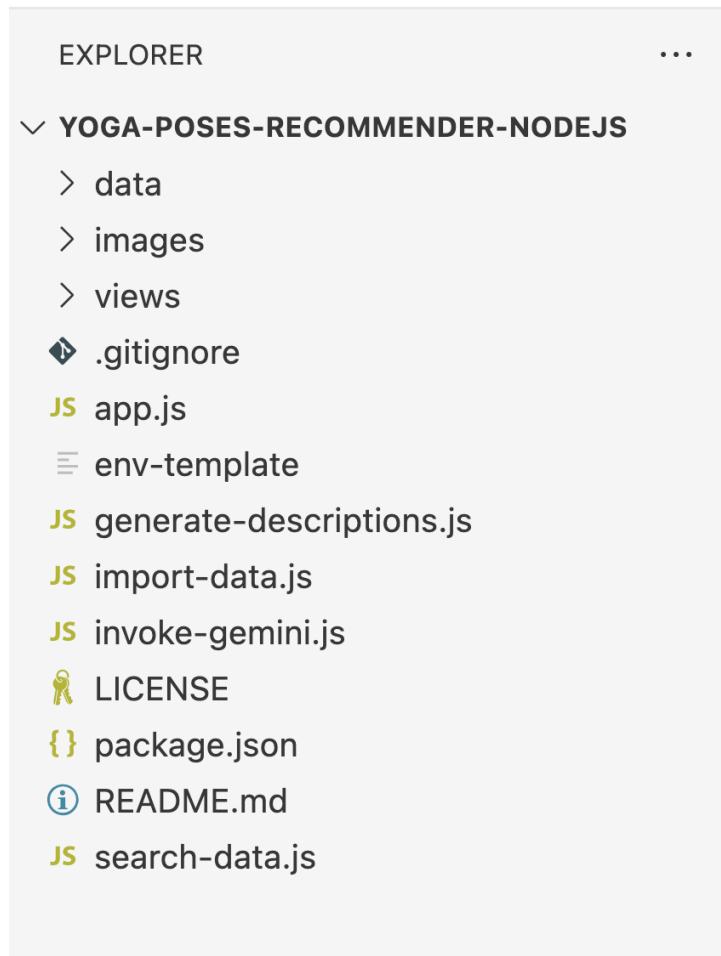
下一步是複製範例存放區，我們會在程式碼研究室的其餘部分參照這個存放區。假設您位於 Cloud Shell 中，請從主目錄執行下列指令：

```
git clone https://github.com/rominirani/yoga-poses-recommender-nodejs
```

如要啟動編輯器，請點選 Cloud Shell 視窗工具列中的「開啟編輯器」。按一下左上角的選單列，然後選取「檔案」→「開啟資料夾」，如下所示：



選取 `yoga-poses-recommender-nodejs` 資料夾，您應該會看到資料夾開啟，並顯示下列檔案，如下所示：



現在需要設定要使用的環境變數。按一下 `env-template` 檔案，您應該會看到如下所示的內容：

```
PROJECT_ID=<YOUR_GOOGLE_CLOUD_PROJECT_ID>
LOCATION=us-<GOOGLE_CLOUD_REGION_NAME>
GEMINI_MODEL_NAME=<GEMINI_MODEL_NAME>
EMBEDDING_MODEL_NAME=<GEMINI_EMBEDDING_MODEL_NAME>
IMAGE_GENERATION_MODEL_NAME=<IMAGEN_MODEL_NAME>
DATABASE=<FIRESTORE_DATABASE_NAME>
COLLECTION=<FIRESTORE_COLLECTION_NAME>
TEST_COLLECTION=test-poses
TOP_K=3
```

請根據您建立 Google Cloud 專案和 Firestore 資料庫區域時選取的項目，更新 `PROJECT_ID` 和 `LOCATION` 的值。理想情況下，Google Cloud 專案和 Firestore 資料庫的 `LOCATION` 值應相同，例如 `us-central1`。

以本次程式碼研究室為例，我們將使用下列值 (當然，`PROJECT_ID` 和 `LOCATION` 除外，您必須根據設定進行設定)。

```
PROJECT_ID=<YOUR_GOOGLE_CLOUD_PROJECT_ID>
LOCATION=us-<GOOGLE_CLOUD_REGION_NAME>
GEMINI_MODEL_NAME=gemini-1.5-flash-002
EMBEDDING_MODEL_NAME=text-embedding-004
IMAGE_GENERATION_MODEL_NAME=imagen-3.0-fast-generate-001
DATABASE=(default)
COLLECTION=poses
TEST_COLLECTION=test-poses
TOP_K=3
```

請將這個檔案儲存為 `.env`，並放在 `env-template` 檔案所在的資料夾中。

前往 Cloud Shell IDE 左上角的主選單，然後點選 **Terminal → New Terminal**。

使用下列指令前往複製的存放區根資料夾：

```
cd yoga-poses-recommender-nodejs
```

透過下列指令安裝 Node.js 依附元件：

```
npm install
```

太棒了！現在一切就緒，可以繼續設定 Firestore 資料庫。

3. 設定 Firestore

Cloud Firestore 是全代管的無伺服器文件資料庫，我們將用來做為應用程式資料的後端。Cloud Firestore 中的資料會以文件集合的形式建構。

初始化 Firestore 資料庫

前往 Cloud 控制台的 [Firestore 頁面](#)。

如果專案先前未初始化 Firestore 資料庫，請點選 **Create Database** 建立 **default** 資料庫。建立資料庫時，請使用下列值：

- Firestore 模式：**Native**。
- 將「位置類型」選為 **Region**，然後選取區域的 **us-central1** 位置。
- 在「安全性規則」部分，選擇 **Test rules**。
- 建立資料庫。

Database ID ↑	Mode	Location
(default)	 Native	us-central1

在下一節中，我們將為在預設 Firestore 資料庫中建立名為 **poses** 的集合奠定基礎。這個集合會保存範例資料 (文件) 或瑜珈姿勢資訊，我們稍後會在應用程式中使用這些資料。

請注意，如果您使用 Firebase，這就是您熟悉且喜愛的 Firestore。事實上，您可以使用 Firebase 控制台存取同一個資料庫執行個體！

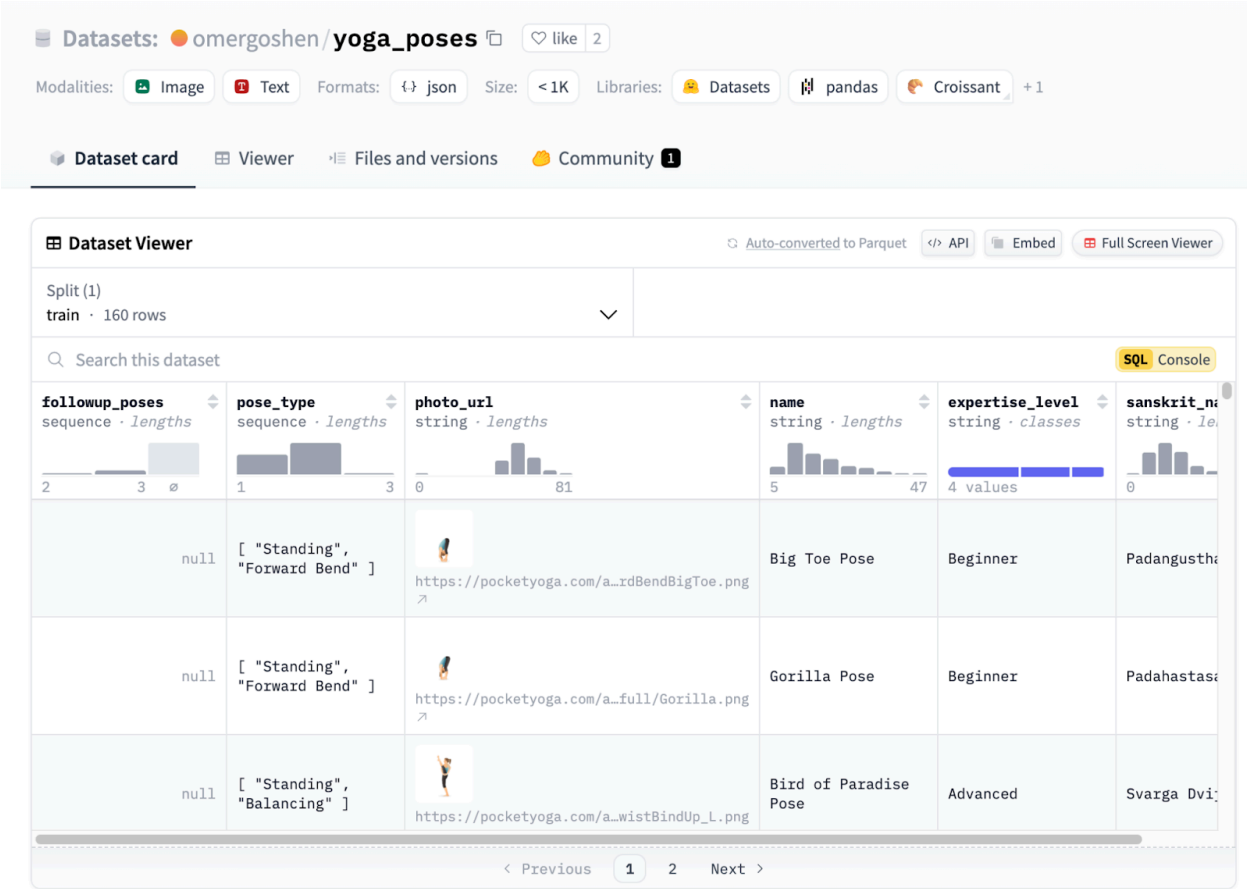
這樣就完成 Firestore 資料庫的設定部分。

步驟 4 · 約 15 分鐘

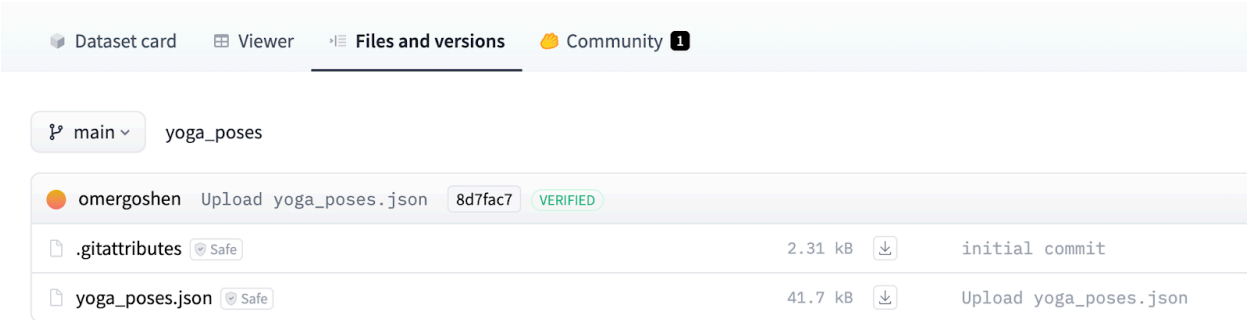
4. 準備 Yoga 姿勢資料集

我們的第一項工作是準備應用程式要使用的 Yoga Poses 資料集。我們會先使用現有的 Hugging Face 資料集，然後加入額外資訊來強化資料集。

請參閱 [Hugging Face 的瑜珈姿勢資料集](#)。請注意，雖然本程式碼研究室使用其中一個資料集，但您其實可以使用任何其他資料集，並按照示範的相同技巧來強化資料集。



前往 **Files and versions** 區段，即可取得所有姿勢的 JSON 資料檔案。



我們已下載 **yoga_poses.json**，並將該檔案提供給您。這個檔案名為 **yoga_poses_alldata.json**，位於 **/data** 資料夾中。

我們會示範生成說明和最終生成嵌入內容的完整程序，並使用一小部分記錄，讓您熟悉程序。

前往 Cloud Shell 編輯器中的 `data/yoga_poses.json` 檔案，查看 JSON 物件清單，其中每個 JSON 物件都代表一個瑜珈姿勢。我們總共有 3 筆記錄，範例記錄如下所示：

```
{
  "name": "Big Toe Pose",
  "sanskrit_name": "Padangusthasana",
  "photo_url": "https://pocketyoga.com/assets/images/full/ForwardBendBigToe.png",
  "expertise_level": "Beginner",
  "pose_type": ["Standing", "Forward Bend"]
}
```

現在是絕佳時機，讓我們介紹 Gemini，以及如何使用預設模型本身來生成 `description` 欄位。

在 Cloud Shell 編輯器中，前往 `generate-descriptions.js` 檔案。這個檔案的內容如下所示：

```

import { VertexAI } from "@langchain/google-vertexai";
import fs from 'fs/promises'; // Use fs/promises for async file operations
import dotenv from 'dotenv';
import pRetry from 'p-retry';
import { promisify } from 'util';

const sleep = promisify(setTimeout);

// Load environment variables
dotenv.config();

async function callGemini(poseName, sanskritName, expertiseLevel, poseTypes) {

  const prompt = `
  Generate a concise description (max 50 words) for the yoga pose: ${poseName}
  Also known as: ${sanskritName}
  Expertise Level: ${expertiseLevel}
  Pose Type: ${poseTypes.join(', ')}

  Include key benefits and any important alignment cues.
  `;

  try {
    // Initialize Vertex AI Gemini model
    const model = new VertexAI({
      model: process.env.GEMINI_MODEL_NAME,
      location: process.env.LOCATION,
      project: process.env.PROJECT_ID,
    });
    // Invoke the model
    const response = await model.invoke(prompt);
    // Return the response
    return response;
  } catch (error) {
    console.error("Error calling Gemini:", error);
    throw error; // Re-throw the error for handling in the calling function
  }
}

// Configure logging (you can use a library like 'winston' for more advanced logging)
const logger = {
  info: (message) => console.log(`INFO - ${new Date().toISOString()} - ${message}`),
  error: (message) => console.error(`ERROR - ${new Date().toISOString()} - ${message}`),
};

async function generateDescription(poseName, sanskritName, expertiseLevel, poseTypes) {
  const prompt = `
  Generate a concise description (max 50 words) for the yoga pose: ${poseName}
  Also known as: ${sanskritName}
  Expertise Level: ${expertiseLevel}
  Pose Type: ${poseTypes.join(', ')}

```

```

    Include key benefits and any important alignment cues.
    `;

const req = {
  contents: [{ role: 'user', parts: [{ text: prompt }] }],
};

const runWithRetry = async () => {
  const resp = await generativeModel.generateContent(req);
  const response = await resp.response;
  const text = response.candidates[0].content.parts[0].text;
  return text;
};

try {
  const text = await pRetry(runWithRetry, {
    retries: 5,
    onFailedAttempt: (error) => {
      logger.info(
        `Attempt ${error.attemptNumber} failed. There are ${error.retriesLeft} retries left.
Waiting ${error.retryDelay}ms...`
      );
    },
    minTimeout: 4000, // 4 seconds (exponential backoff will adjust this)
    factor: 2, // Exponential factor
  });
  return text;
} catch (error) {
  logger.error(`Error generating description for ${poseName}: ${error}`);
  return '';
}
}

async function addDescriptionsToJSON(inputFile, outputFile) {
  try {
    const data = await fs.readFile(inputFile, 'utf-8');
    const yogaPoses = JSON.parse(data);

    const totalPoses = yogaPoses.length;
    let processedCount = 0;

    for (const pose of yogaPoses) {
      if (pose.name !== ' Pose') {
        const startTime = Date.now();
        pose.description = await callGemini(
          pose.name,
          pose.sanskrit_name,
          pose.expertise_level,
          pose.pose_type
        );

        const endTime = Date.now();

```

```

        const timeTaken = (endTime - startTime) / 1000;
        processedCount++;
        logger.info(`Processed: ${processedCount}/${totalPoses} - ${pose.name}
(${timeTaken.toFixed(2)} seconds)`);
    } else {
        pose.description = '';
        processedCount++;
        logger.info(`Processed: ${processedCount}/${totalPoses} - ${pose.name}
(${timeTaken.toFixed(2)} seconds)`);
    }

    // Add a delay to avoid rate limit
    await sleep(30000); // 30 seconds
}

await fs.writeFile(outputFile, JSON.stringify(yogaPoses, null, 2));
logger.info(`Descriptions added and saved to ${outputFile}`);
} catch (error) {
    logger.error(`Error processing JSON file: ${error}`);
}
}

async function main() {
    const inputFile = './data/yoga_poses.json';
    const outputFile = './data/yoga_poses_with_descriptions.json';

    await addDescriptionsToJSON(inputFile, outputFile);
}

main();

```

這個應用程式會在每個 Yoga 姿勢 JSON 記錄中新增 **description** 欄位。這項功能會呼叫 Gemini 模型，並提供必要的提示，藉此取得說明。欄位會新增至 JSON 檔案，而新檔案會寫入 **data/yoga_poses_with_descriptions.json** 檔案。

主要步驟如下：

1. 在 **main()** 函式中，您會發現它會叫用 **add_descriptions_to_json** 函式，並提供輸入檔案和預期的輸出檔案。
2. **add_descriptions_to_json** 函式會針對每個 JSON 記錄 (即瑜珈貼文資訊) 執行下列作業：
3. 並擷取 **pose_name**、**sanskrit_name**、**expertise_level** 和 **pose_types**。
4. 這個函式會叫用 **callGemini** 函式建構提示詞，然後叫用 LangchainVertexAI 模型類別來取得回覆文字。
5. 然後將這段回覆文字新增至 JSON 物件。
6. 更新後的物件 JSON 清單隨即會寫入目的地檔案。

讓我們執行這個應用程式。啟動新的終端機視窗 (Ctrl+Shift+C)，然後輸入下列指令：

```
npm run generate-descriptions
```

如果系統要求授權，請提供授權。

您會發現應用程式開始執行。為避免新 Google Cloud 帳戶可能出現任何速率限制配額，我們在記錄之間新增了 30 秒的延遲時間，請耐心等待。

下方顯示執行中的範例：

```
INFO - 2025-01-28T06:52:40.528Z - Processed: 1/3 - Big Toe Pose (8.69 seconds)
INFO - 2025-01-28T06:53:13.104Z - Processed: 2/3 - Gorilla Pose (2.57 seconds)
INFO - 2025-01-28T06:53:46.783Z - Processed: 3/3 - Bird of Paradise Pose (3.65 seconds)
INFO - 2025-01-28T06:54:16.808Z - Descriptions added and saved to ./data/yoga_poses_with_descriptions.json
```

使用 Gemini 呼叫功能強化所有 3 筆記錄後，系統會生成 `data/yoga_poses_with_description.json` 檔案。你可以參考這篇文章。

現在我們已準備好資料檔案，下一步是瞭解如何使用該檔案填入 Firestore 資料庫，以及如何產生嵌入。

5. 將資料匯入 Firestore 並生成向量嵌入

我們已有 `data/yoga_poses_with_description.json` 檔案，現在需要使用該檔案填入 Firestore 資料庫，並為每筆記錄產生向量嵌入。稍後，當我們必須對使用者以自然語言提供的查詢執行相似度搜尋時，向量嵌入就會派上用場。

具體操作步驟如下：

1. 我們會將 JSON 物件清單轉換為物件清單。每個文件都會有兩個屬性：`content` 和 `metadata`。中繼資料物件會包含整個 JSON 物件，其中具有 `name`、`description`、`sanskrit_name` 等屬性。`content` 會是字串文字，由幾個欄位串連而成。
2. 取得文件清單後，我們將使用 Vertex AI Embeddings 類別，為內容欄位生成嵌入。這項嵌入內容會新增至每份文件記錄，然後我們會使用 Firestore API 將這份文件物件清單儲存在集合中 (我們使用的是指向 `test-poses` 的 `TEST_COLLECTION` 變數)。

`import-data.js` 的程式碼如下所示 (為求簡潔，部分程式碼已截斷)：

```

import { Firestore,
        FieldValue,
    } from '@google-cloud/firestore';
import { VertexAIEmbeddings } from '@langchain/google-vertexai';
import * as dotenv from 'dotenv';
import fs from 'fs/promises';

// Load environment variables
dotenv.config();

// Configure logging
const logger = {
    info: (message) => console.log(`INFO - ${new Date().toISOString()} - ${message}`),
    error: (message) => console.error(`ERROR - ${new Date().toISOString()} - ${message}`),
};

async function loadYogaPosesDataFromLocalFile(filename) {
    try {
        const data = await fs.readFile(filename, 'utf-8');
        const poses = JSON.parse(data);
        logger.info(`Loaded ${poses.length} poses.`);
        return poses;
    } catch (error) {
        logger.error(`Error loading dataset: ${error}`);
        return null;
    }
}

function createFirestoreDocuments(poses) {
    const documents = [];
    for (const pose of poses) {
        // Convert the pose to a string representation for pageContent
        const pageContent = `
name: ${pose.name || ''}
description: ${pose.description || ''}
sanskrit_name: ${pose.sanskrit_name || ''}
expertise_level: ${pose.expertise_level || 'N/A'}
pose_type: ${pose.pose_type || 'N/A'}
`.trim();

        // The metadata will be the whole pose
        const metadata = pose;
        documents.push({ pageContent, metadata });
    }
    logger.info(`Created ${documents.length} Langchain documents.`);
    return documents;
}

async function main() {
    const allPoses = await
loadYogaPosesDataFromLocalFile('./data/yoga_poses_with_descriptions.json');
    const documents = createFirestoreDocuments(allPoses);

```



```

    logger.info(`Successfully created Firestore documents. Total documents:
    ${documents.length}`);

    const embeddings = new VertexAIEmbeddings({
      model: process.env.EMBEDDING_MODEL_NAME,
    });
    // Initialize Firestore
    const firestore = new Firestore({
      projectId: process.env.PROJECT_ID,
      databaseId: process.env.DATABASE,
    });

    const collectionName = process.env.TEST_COLLECTION;

    for (const doc of documents) {
      try {
        // 1. Generate Embeddings
        const singleVector = await embeddings.embedQuery(doc.pageContent);

        // 2. Store in Firestore with Embeddings
        const firestoreDoc = {
          content: doc.pageContent,
          metadata: doc.metadata, // Store the original data as metadata
          embedding: FieldValue.vector(singleVector), // Add the embedding vector
        };

        const docRef = firestore.collection(collectionName).doc();
        await docRef.set(firestoreDoc);
        logger.info(`Document ${docRef.id} added to Firestore with embedding.`);
      } catch (error) {
        logger.error(`Error processing document: ${error}`);
      }
    }

    logger.info('Finished adding documents to Firestore.');
```

```

main();

```

讓我們執行這個應用程式。啟動新的終端機視窗 (Ctrl+Shift+C)，然後輸入下列指令：

```
npm run import-data
```

如果一切順利，您應該會看到類似以下的訊息：

```
INFO - 2025-01-28T07:01:14.463Z - Loaded 3 poses.
INFO - 2025-01-28T07:01:14.464Z - Created 3 Langchain documents.
INFO - 2025-01-28T07:01:14.464Z - Successfully created Firestore documents. Total documents:
3
INFO - 2025-01-28T07:01:17.623Z - Document P46d5F92z9FsIhVVYgkd added to Firestore with
embedding.
INFO - 2025-01-28T07:01:18.265Z - Document bjXXISctkXl2ZRSjUYVR added to Firestore with
embedding.
INFO - 2025-01-28T07:01:19.285Z - Document GwzZMzyPfTLtiX6qBFFz added to Firestore with
embedding.
INFO - 2025-01-28T07:01:19.286Z - Finished adding documents to Firestore.
```

如要確認記錄是否已順利插入，以及是否已產生嵌入內容，請前往 Cloud 控制台的 [Firestore 頁面](#)。

Filter		
Database ID 	Mode	Location
(default)	 Native	asia-south1

按一下 (預設) 資料庫，這時應該會顯示 **test-poses** 集合，以及該集合下的多個文件。每份文件都是一個瑜珈姿勢。

All databases > DATABASE (default) ⓘ		
PANEL VIEW QUERY BUILDER		
/ > test-poses > GwzZMzyPfTLtiX6qBFFz ✎		
(default)	test-poses ⌵ ⋮	GwzZMzyPfTLtiX6qBFFz ⋮
+ START COLLECTION	+ ADD DOCUMENT	+ START COLLECTION
test-poses >	GwzZMzyPfTLtiX6qBFFz >	+ ADD FIELD
	P46d5F92z9FsIhVVYgkd	content: "name: Bird of Paradise Pose description: ## Bird of P...
	bjXXISctkXl2ZRSjUYVR	embedding: vector<768> ⓘ
		▼ metadata
		description: "## Bird of Paradise Pose (Svarga Dvijasana) ...
		expertise_level: "Advanced"
		name: "Bird of Paradise Pose"
		photo_url: "https://pocketyoga.com/assets/images/full/Ch...
		▼ pose_type
		0: "Standing"
		1: "Balancing"
		sanskrit_name: "Svarga Dvijasana"

點選任一文件即可調查欄位。除了匯入的欄位，您也會看到 **embedding** 欄位，這是向量欄位，其值是透過 **text-embedding-004** Vertex AI 嵌入模型產生。

GwzMZyPftLtIX6qBFFz	⋮
+ START COLLECTION	
+ ADD FIELD	
content: "name: Bird of Paradise Pose description: ## Bird of P...	
embedding: vector<768> ⓘ	
▼ metadata	
description: "## Bird of Paradise Pose (Svarga Dvijasana) ...	
expertise_level: "Advanced"	
name: "Bird of Paradise Pose"	
photo_url: "https://pocketyoga.com/assets/images/full/Ch...	
▼ pose_type	
0: "Standing"	
1: "Balancing"	
sanskrit_name: "Svarga Dvijasana"	

現在記錄已上傳至 Firestore 資料庫，且嵌入項目也已就位，我們可以前往下一個步驟，瞭解如何在 Firestore 中執行向量相似度搜尋。

6. 將完整瑜珈姿勢匯入 Firestore 資料庫集合

現在我們要建立 `poses` 集合，其中包含 160 個瑜珈姿勢的完整清單。我們已產生資料庫匯入檔案，您可以直接匯入。這是為了節省實驗室時間。產生包含說明和嵌入內容的資料庫時，所用的程序與上一節所述相同。

請按照下列步驟匯入資料庫：

- 使用下列 `gsutil` 指令，在專案中建立 bucket。在下方指令中，將 `<PROJECT_ID>` 變數替換為您的 Google Cloud 專案 ID。

```
gsutil mb -l us-central1 gs://<PROJECT_ID>-my-bucket
```

- 建立好 bucket 後，我們需要將準備好的資料庫匯出內容複製到這個 bucket，才能匯入 Firebase 資料庫。使用下列指令：

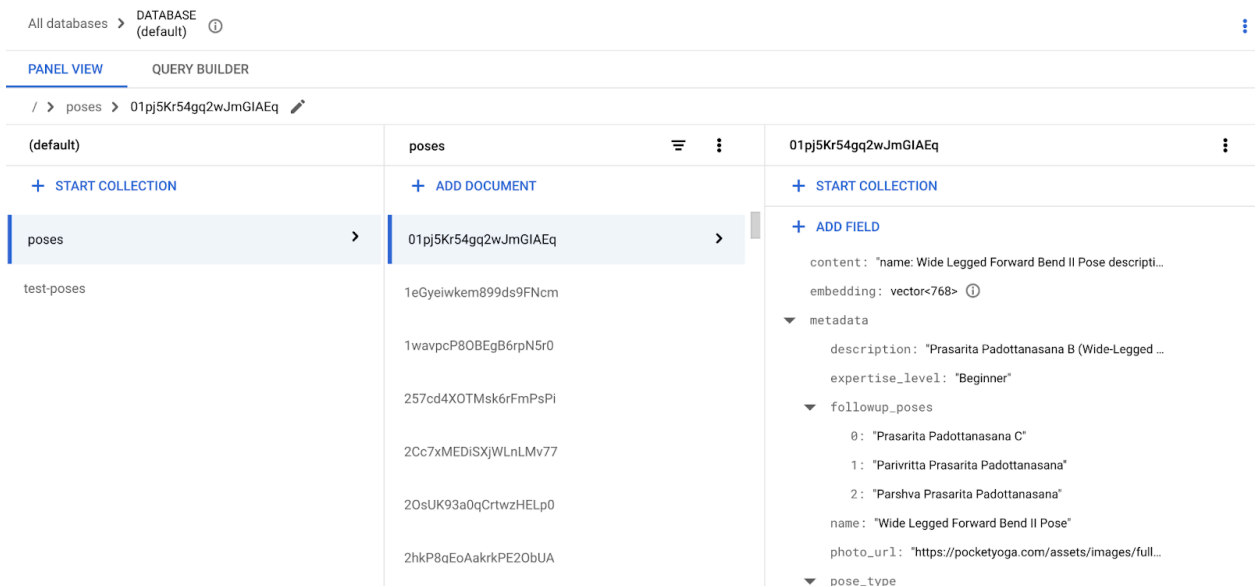
```
gsutil cp -r gs://yoga-database-firestore-export-bucket/2025-01-27T05:11:02_62615
gs://<PROJECT_ID>-my-bucket
```

現在我們已準備好要匯入的資料，可以進入最後一個步驟，將資料匯入我們建立的 Firebase 資料庫 (`default`)。

- 使用下列 `gcloud` 指令：

```
gcloud firestore import gs://<PROJECT_ID>-my-bucket/2025-01-27T05:11:02_62615
```

匯入作業需要幾秒鐘，完成後請前往 <https://console.cloud.google.com/firestore/databases>，選取 `default` 資料庫和 `poses` 集合，驗證 Firestore 資料庫和集合，如下所示：



這樣就完成了要在應用程式中使用的 Firestore 集合建立作業。

7. 在 Firestore 中執行向量相似度搜尋

如要執行[向量相似度搜尋](#)，我們會接收使用者的查詢。這類查詢的範例為 **"Suggest me some exercises to relieve back pain"**。

請查看 `search-data.js` 檔案。要查看的主要函式是 `search` 函式，如下所示。從高層次來看，這會建立嵌入類別，用於產生使用者查詢的嵌入內容。接著，它會建立與 Firestore 資料庫和集合的連線。然後在集合上叫用 `findNearest` 方法，執行向量相似度搜尋。

```
async function search(query) {
  try {

    const embeddings = new VertexAIEmbeddings({
      model: process.env.EMBEDDING_MODEL_NAME,
    });

    // Initialize Firestore
    const firestore = new Firestore({
      projectId: process.env.PROJECT_ID,
      databaseId: process.env.DATABASE,
    });

    log.info(`Now executing query: ${query}`);
    const singleVector = await embeddings.embedQuery(query);

    const collectionRef = firestore.collection(process.env.COLLECTION);
    let vectorQuery = collectionRef.findNearest(
      "embedding",
      FieldValue.vector(singleVector), // a vector with 768 dimensions
    {
      limit: process.env.TOP_K,
      distanceMeasure: "COSINE",
    }
    );
    const vectorQuerySnapshot = await vectorQuery.get();

    for (const result of vectorQuerySnapshot.docs) {
      console.log(result.data().content);
    }
  } catch (error) {
    log.error(`Error during search: ${error.message}`);
  }
}
```

使用幾個查詢範例執行這項作業前，請先產生 [Firestore 複合式索引](#)，這是查詢作業成功執行的必要條件。如果您在未建立索引的情況下執行應用程式，系統會顯示錯誤訊息，指出您必須先建立索引，並提供建立索引的指令。

建立複合索引的 `gcloud` 指令如下：

```
gcloud firestore indexes composite create --project=<YOUR_PROJECT_ID> --collection-group=poses --query-scope=COLLECTION --field-config=vector-config='{ "dimension": "768", "flat": "{}" }', field-path=embedding
```

由於資料庫中有 150 筆以上的記錄，因此建立索引需要幾分鐘的時間。完成後，您可以使用下列指令查看索引：

```
gcloud firestore indexes composite list
```

清單中應該會顯示您剛建立的索引。

現在請試試下列指令：

```
node search-data.js --prompt "Recommend me some exercises for back pain relief"
```

系統會提供幾項建議。以下是執行範例：

```
2025-01-28T07:09:05.250Z - INFO - Now executing query: Recommend me some exercises for back pain relief
name: Sphinx Pose
description: A gentle backbend, Sphinx Pose (Salamba Bhujangasana) strengthens the spine and opens the chest. Keep shoulders relaxed, lengthen the tailbone, and engage the core for optimal alignment. Beginner-friendly.

sanskrit_name: Salamba Bhujangasana
expertise_level: Beginner
pose_type: ['Prone']
name: Supine Spinal Twist Pose
description: A gentle supine twist (Supta Matsyendrasana), great for beginners. Releases spinal tension, improves digestion, and calms the nervous system. Keep shoulders flat on the floor and lengthen your spine throughout the twist.

sanskrit_name: Supta Matsyendrasana
expertise_level: Beginner
pose_type: ['Supine', 'Twist']
name: Reverse Corpse Pose
description: Reverse Corpse Pose (Advasana) is a beginner prone pose. Lie on your belly, arms at your sides, relaxing completely. Benefits include stress release and spinal decompression. Ensure your forehead rests comfortably on the mat.

sanskrit_name: Advasana
expertise_level: Beginner
pose_type: ['Prone']
```

完成上述步驟後，您就瞭解如何使用 Firestore 向量資料庫上傳記錄、產生嵌入內容，以及執行向量相似度搜尋。現在可以建立網頁應用程式，將向量搜尋整合至網路前端。

步驟 8 · 約 20 分鐘

8. 網頁應用程式

Python Flask 網頁應用程式位於 `app.js` 檔案中，前端 HTML 檔案則位於 `views/index.html`。

建議您查看這兩個檔案。首先，請從包含 `/search` 處理常式的 `app.js` 檔案開始，該處理常式會接收從前端 HTML `index.html` 檔案傳遞的提示。接著，這會叫用搜尋方法，執行上一節中介紹的向量相似度搜尋。

接著，系統會將回應連同建議清單傳回給 `index.html`。 `index.html` 接著會以不同資訊卡的形式顯示建議。

在本機執行應用程式

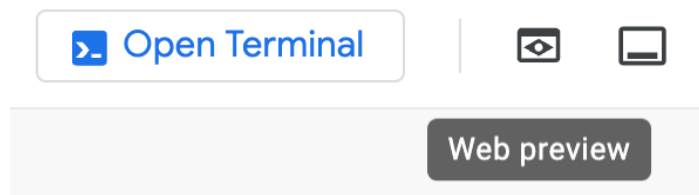
啟動新的終端機視窗 (Ctrl+Shift+C) 或任何現有的終端機視窗，然後輸入下列指令：

```
npm run start
```

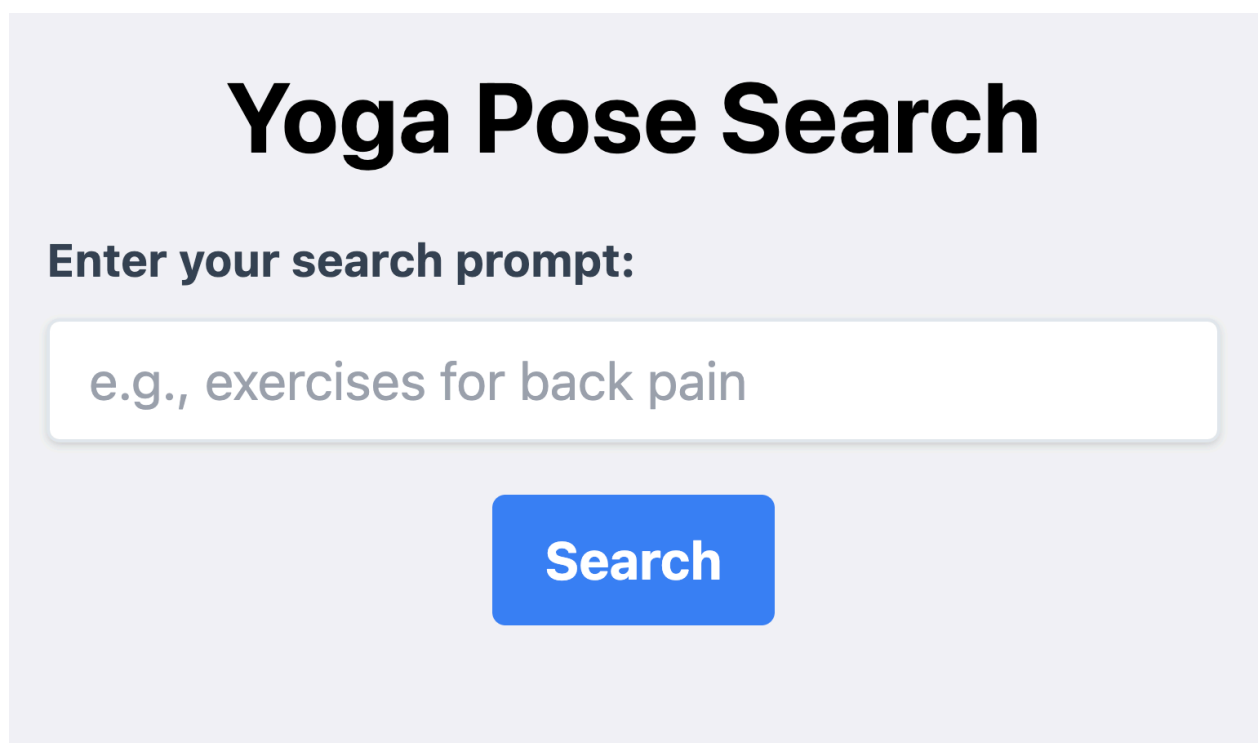
執行範例如下所示：

```
...  
Server listening on port 8080
```

啟動並執行後，按一下下方顯示的「網頁預覽」按鈕，即可前往應用程式的首頁網址：



如下所示，畫面應會顯示所提供的 `index.html` 檔案：



提供範例查詢 (例如：**Provide me some exercises for back pain relief**)，然後按一下「**Search**」按鈕。這應該會從資料庫擷取一些建議。你也會看到 **Play Audio** 按鈕，點選後系統會根據說明生成音訊串流，你可以直接聆聽。

Yoga Pose Search

Enter your search prompt:

Provide exercises for back pain

Search



Bridge Pose

Bridge Pose (Setu Bandha Sarvangasana) strengthens back, glutes, and hamstrings. Press into feet, lift hips, and keep thighs parallel. Lengthen spine, avoid neck strain.

Expertise: Intermediate

Supine

Back Bend

Play Audio



Locust I Pose

Locust Pose I (Shalabhasana A) strengthens the back, glutes, and shoulders. Lie prone, lift chest and legs simultaneously, engaging back muscles. Keep hips grounded and gaze slightly forward.

Expertise: Intermediate

Prone

Back Bend

Play Audio



Camel Pose

Camel Pose (Ushtrasana) is an intermediate backbend opening the chest and hips. Maintain a neutral neck, engage core, and gently arch the spine, avoiding hyperextension. Benefits include improved posture and spinal flexibility.

Expertise: Intermediate

Arm Leg Support

Back Bend

Play Audio

步驟 9 · 約 10 分鐘

9. (選用) 部署至 Google Cloud Run

最後一個步驟是將這個應用程式部署至 Google Cloud Run。部署指令如下所示，請務必在部署前，將括號 (<<>>) 中的各種值替換成下方顯示的值。這些值可從 `.env` 檔案擷取。

```
gcloud run deploy yogaposes --source . \
  --port=8080 \
  --allow-unauthenticated \
  --region=<<YOUR_LOCATION>> \
  --platform=managed \
  --project=<<YOUR_PROJECT_ID>> \
  --set-env-vars=PROJECT_ID="<<YOUR_PROJECT_ID>>",LOCATION="
<<YOUR_LOCATION>>","EMBEDDING_MODEL_NAME="<<EMBEDDING_MODEL_NAME>>","DATABASE="
<<FIRESTORE_DATABASE_NAME>>","COLLECTION="<<FIRESTORE_COLLECTION_NAME>>","TOP_K=
<<YOUR_TOP_K_VALUE>>
```

從應用程式的根資料夾執行上述指令。系統也可能會要求您啟用 Google Cloud API，並確認各種權限，請按照指示操作。

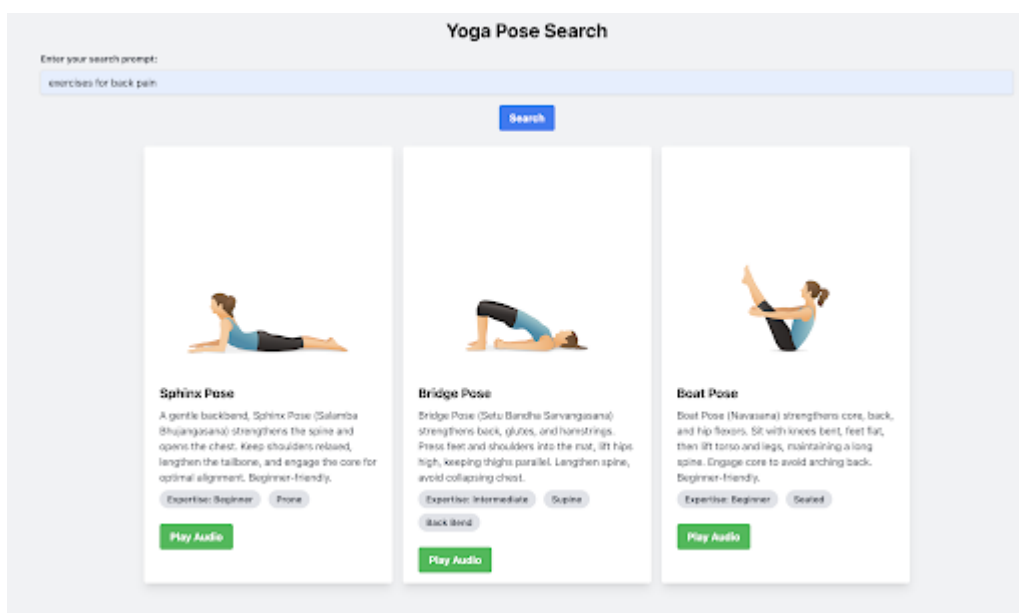
部署程序大約需要 5 到 7 分鐘才能完成，請耐心等待。

```
Building using Buildpacks and deploying container to Cloud Run service [yogaposes] in project [build-with-ai-tour-testing-2] region [us-central1]
OK Building and deploying new service... Done.
OK Creating Container Repository...
OK Uploading sources...
OK Building Container... Logs are available at [https://console.cloud.google.com/cloud-build/builds/1d73a3d6-18e5-446f-b8a8-842279ff6aaf?project=1068063135560].
OK Creating Revision...
OK Routing traffic...
OK Setting IAM Policy...
Done.
Service [yogaposes] revision [yogaposes-00001-xzt] has been deployed and is serving 100 percent of traffic.
Service URL: https://yogaposes-1068063135560.us-central1.run.app
```

成功部署後，部署輸出內容會提供 Cloud Run 服務網址。這是測試。

Service URL: <https://yogaposes-<UNIQUEID>.us-central1.run.app>

前往該公開網址，您應該會看到已部署且成功執行的相同網頁應用程式。



您也可以從 Google Cloud 控制台前往 [Cloud Run](#)，查看 Cloud Run 中的服務清單。`yogaposes` 服務應是列於該處的服務之一 (如果不是唯一服務)。

Cloud Run

Services

DEPLOY CONTAINER

CONNECT REPO

WRITE A FUNCTION

MANAGE CUSTOM DOMAINS

SERVICES

JOB

A service exposes a unique endpoint and automatically scales the underlying infrastructure to handle incoming requests.
Deploy a container image, source code or a function to create a service.

Services

Filter Filter services

	Name	Deployment type	Req/sec	Region	Authentication	Ingress	Recommendation	Last deployed
	yogaposes	Container	0	us-central1	Allow unauthenticated	All	—	1 minute ago

按一下特定服務名稱 (在本例中為 **yogaposes**)，即可查看服務詳細資料，例如網址、設定、記錄等。

Cloud Run

Service details

EDIT AND DEPLOY NEW REVISION

SET UP CONTINUOUS DEPLOYMENT

TEST

yogaposes

Region: us-central1

URL: https://yogaposes-1068063135560-us-central1.run.app

Service min instances: 0

METRICS

SLOS

LOGS

REVISIONS

SOURCE

TRIGGERS

NETWORKING

SECURITY

YAML

Predefined

CREATE UPTIME CHECK

Annotations (2)

Last day

No errors found during this interval.

See more in Error reporting

Request count

No data is available for the selected time frame.

UTC+5:30 18:00 20:00 22:00 27 Jan 02:00 04:00 06:00 08:00 10:00 12:00

0 time series

Request latencies

No data is available for the selected time frame.

UTC+5:30 18:00 20:00 22:00 27 Jan 02:00 04:00 06:00 08:00 10:00

0 time series

Container instance count

1

Billable container instance time

這樣就完成了在 Cloud Run 上開發及部署瑜珈姿勢建議網頁應用程式。

步驟 10 · 約 0 分鐘

10. 恭喜

恭喜！您已成功建構應用程式，可將資料集上傳至 Firestore、產生嵌入項目，並根據使用者查詢執行向量相似度搜尋。

參考文件

- [Firestore](#)
 - [Firestore - Search with Vector Embeddings](#)
 - [管理 Firestore 中的索引](#)
 - [Cloud Run](#)
-